

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): студент(-ка) групи AI-243
Плотнікова І.С.

Перевірів: ас. Баранов О.М.

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	310	5
Час, с	37,5	27,6
Токенів/с	122,7	149,2

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	0 / 5
eval_count	1390,8	146,8
Час, с	12, 15	1,31

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Ці додаткові токени у міркувальній моделі (qwen3:4b) витрачаються на генерацію прихованого внутрішнього монологу (так званого «ланцюжка думок» або Chain of Thought), під час якого неймережа детально аналізує питання, міркує над контекстом та перевіряє власні припущення перед тим, як сформулювати фінальний результат. На відміну від інструктивної моделі (llama3.2:3b), яка натренована максимально швидко видавати пряму відповідь, міркувальна модель

завжди витрачає обчислювальні ресурси на попередній крок самоаналізу. Для базових фактичних питань це призводить до надмірної генерації токенів, проте саме цей механізм дозволяє таким моделям успішно розв'язувати складні логічні та математичні задачі, де пряме вгадування не працює.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі дають значну перевагу під час вирішення задач, які вимагають багатоетапного логічного аналізу, складних математичних обчислень, написання алгоритмічного коду або розв'язання нестандартних проблем (як-от задача із зошитом і ручкою), оскільки завдяки попередньому «обдумуванню» вони здатні уникати поверхневих, інтуїтивних, але хибних висновків. Натомість їхнє використання є абсолютно невиправданим та ресурсозатратним для простих фактологічних запитів, швидкого перекладу, форматування тексту чи базового пошуку інформації, де інструктивні моделі справляються в десятки разів швидше, не витрачаючи час та токени на непотрібний внутрішній монолог.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Процес навчання інструктивних моделей базується переважно на контрольованому тонкому налаштуванні (Supervised Fine-Tuning) та навчанні з підкріпленням відгуками людей (RLHF), де головна мета — навчити нейромережу максимально швидко, точно та безпечно виконувати пряму вказівку, генеруючи одразу кінцевий результат. Натомість навчання міркувальних моделей робить особливий акцент на навчанні з підкріпленням, де модель винагороджується не лише за правильну фінальну відповідь, але й за логічність, правильність та повноту проміжних кроків (Chain of Thought). Простіше кажучи, якщо інструктивну модель цілеспрямовано вчать «як правильно відповісти», то міркувальну тренують «як правильно думати» крок за кроком перед тим, як сформулювати остаточний висновок.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість згенерованих токенів не є прямим показником того, що міркувальна модель «розумніша» за інструктивну, адже це лише свідчить про використання іншого алгоритмічного підходу до обробки інформації.

Додаткові токени витрачаються на експліцитне розгортання проміжних логічних кроків, що допомагає моделі розв'язувати складні багатокрокові задачі, але сама база знань чи здатність оперувати фактами в обох архітектурах може бути абсолютно однаковою. Інструктивна модель може володіти таким самим або навіть більшим обсягом знань, але вона натренована видавати результат миттєво, тоді як міркувальна може згенерувати тисячі токенів «роздумів» і все одно дійти до помилкового висновку, якщо її логічний ланцюжок виявиться хибним. Тому обсяг токенів відображає лише деталізацію процесу пошуку рішення та глибину самоаналізу, а не загальний рівень «інтелекту» чи ерудованості нейромережі.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різниця у швидкості генерації (кількості токенів за секунду) між цими двома моделями пояснюється насамперед їхнім архітектурним розміром, тобто кількістю параметрів. Модель qwen3:4b має 4 мільярди параметрів, тоді як Llama3.2:3b є «легшою» і містить 3 мільярди. Під час генерації кожного окремого токена комп'ютеру необхідно виконати математичні обчислення, пропустивши дані через усі ці внутрішні зв'язки, тому робота з більшою нейромережею вимагає більше обчислювальних ресурсів. Відповідно, на одному й тому ж апаратному забезпеченні менша модель Llama3.2:3b працюватиме помітно швидше і генеруватиме більше токенів за секунду, оскільки фізичне навантаження на процесор під час її роботи є меншим.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі (тому легша Llama3.2:3b генерує токени швидше за qwen3:4b), другий – обсяг роботи, яку модель собі призначила (через довжину внутрішнього монологу).

На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, але часові витрати відрізняються майже у 14 разів (близько 37,5 с проти 2,7 с). На задачі про ручку й зошит ситуація протилежна – саме

розписаний хід думок дозволив міркувальній моделі уникнути інтуїтивно привабливої, але неправильної відповіді, яку інструктивна модель вперто видавала 5 разів із 5.

Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами (на базі середнього часу з логічної задачі) різниця складе 97,2 с (для міркувальної) проти 10,5 с (для інструктивної), що робить міркувальну модель занадто повільною для швидкого налагодження коду.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Варіант 7: “Поясни різницю між процесом і потоком.”

Таблиця 3 - Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано? (Так/Ні)
llama3.2:3b	лаконічний асистент	575	Українська (з англ. словами і русизмами)	5	Не лаконічно
llama3.2:3b	українською, 2 речення	209	Українська(з помилками)	2	Так
qwen3:4b	лаконічний асистент	231	Українська	3	Так
qwen3:4b	українською, 2 речення	146	Українська	2	Так

Таблиця 4 - Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	50	178	228

llama3.2:3b	українською, 2 речення	54	58	112
qwen3:4b	лаконічний асистент	53	2814	2867
qwen3:4b	українською, 2 речення	62	696	758

Відповіді на питання:

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»? Обидві моделі змогли виконати вимогу «максимум двома реченнями», проте qwen3:4b впоралась ідеально і видала чіткий текст. Llama3.2:3b також видала 2 речення, але через свій менший обсяг знань згенерувала граматично кострубатий текст («на противоречу», «процесм»).
2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку? Прямої відмови або переходу на іншу мову не було в жодній моделі. Але варто відзначити, що Llama3.2:3b у першому тесті вставила англійські слова («another», «specific task») та русизми, тоді як qwen3:4b показала ідеальну українську термінологію.
3. Як змінився prompt_tokens при зміні system-повідомлення і чому? Значення prompt_tokens трохи збільшилося (з 50 до 54 для Llama, і з 53 до 62 для qwen). Це сталося через те, що нове system-повідомлення («Відповідай лише українською, максимум двома реченнями.») містить більше символів і слів, ніж перше, тому модель витратила більше токенів на його "прочитання".
4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у Llama3.2:3b, навіть при однаковій довжині видимої відповіді? Модель qwen3:4b є міркувальною моделлю. Перед тим як видати фінальну коротку відповідь, вона генерує великий прихований внутрішній монолог (chain of thought), щоб проаналізувати запит та виконати інструкцію. Тисячі витрачених completion_tokens - це саме ті невидимі роздуми моделі, які користувач не бачить на екрані.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	qwen/qwen3.8-27b
Час відповіді, с	2.85	0.85
completion_tokens	83	329
Довжина відповіді, символів	265	739
Мова відповіді	Укр (з помилками і русизмами)	Ідеальна українська
Фактичних помилок у відповіді	Багато (відповідь не про ІТ-сферу)	Немає

Відповіді на запитання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну? Довелося змінити лише кілька рядків (базову URL-адресу `base_url`, API-ключ `api_key` та назву моделі `model`). Оскільки і локальна Ollama, і хмарний Groq використовують однаковий формат запитів (OpenAI API), логіка самого коду взагалі не змінилася.
2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці. Хмарна відповідь набагато точніша і повніша. Наприклад, локальна модель написала абстрактну нісенітницю («Потік — це набір явищ з певного джерела...»), тоді як хмарна дала чітке ІТ-визначення і навела гарну аналогію: «Процес — це офіс, а потік — це працівник всередині офісу».
3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою? Так, локальна модель зробила фактичну помилку, оскільки її відповідь взагалі не стосувалася комп'ютерних наук (операційних систем). Також були проблеми з мовою: русизми та неіснуючі слова («Принципови», «фіксованими», «или»).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет? Хмарна модель працює на потужних спеціалізованих серверах (у випадку Groq — на тензорних LPU-процесорах), які генерують текст блискавично. Затримка передачі даних через інтернет (пінг) становить мілісекунди, що повністю перекривається величезною швидкістю генерації токенів на сервері.

Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери? Бібліотека та API від OpenAI з'явилися одними з перших і стали найпопулярнішими серед розробників під час буму ChatGPT. Іншим провайдерам (як Ollama чи Groq) було комерційно вигідно зробити свої системи сумісними з цим форматом, щоб розробники могли переходити до них, змінюючи лише один рядок коду (URL-адресу).

2. Наскільки розмір моделі (3 млрд проти 27/70 млрд) впливає на якість відповіді? Розмір моделі є ключовим фактором, оскільки більша кількість параметрів дозволяє моделі запам'ятати більше фактів, краще розуміти складні концепції (як IT-термінологія) та ідеально володіти мовами з меншою кількістю даних в інтернеті (наприклад, українською). Маленькі моделі (3 млрд) гарно підходять для простих задач (наприклад, форматування тексту), але часто "галюцинують" на складних питаннях.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється? Це пояснюється апаратним забезпеченням: звичайні домашні комп'ютери обробляють LLM-моделі на центральному процесорі (CPU) або слабких відеокартах, що дає швидкість 10-20 токенів на секунду. Сервери хмарних провайдерів використовують кластери з найсучасніших чипів, які можуть генерувати сотні або тисячі токенів за секунду, роблячи затримку мережі непомітною.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах? Головний ризик — передача

конфіденційних даних (коду компанії, персональних даних клієнтів) на сторонні сервери, де вони можуть бути перехоплені або використані для навчання інших моделей. У комерційних проєктах цю проблему вирішують підписанням угод про нерозголошення (B2B Enterprise-тарифи, які гарантують відмову від тренування на даних), розгортанням локальних моделей на власних серверах, або маскуванням конфіденційних даних перед відправкою.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження? Rate limit — це штучне обмеження на кількість запитів або згенерованих токенів за хвилину/день (щоб один користувач не перевантажив сервер). Розробники обходять його кількома шляхами: роблять паузи між запитами (time.sleep), автоматично перемикаються між різними API-ключами або провайдерами (fallback), або просто купують платний тариф.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.086	2.234	13.4
локальна	llama3.2:3b	середній	2.066	2.514	81.9
локальна	llama3.2:3b	довгий	2.086	6.829	321.9
хмарна	qwen3.8-27b	короткий	0.211	0.308	94.2
хмарна	qwen3.8-27b	середній	0.306	0.620	493.4
хмарна	qwen3.8-27b	довгий	20.503	22.624	120.9

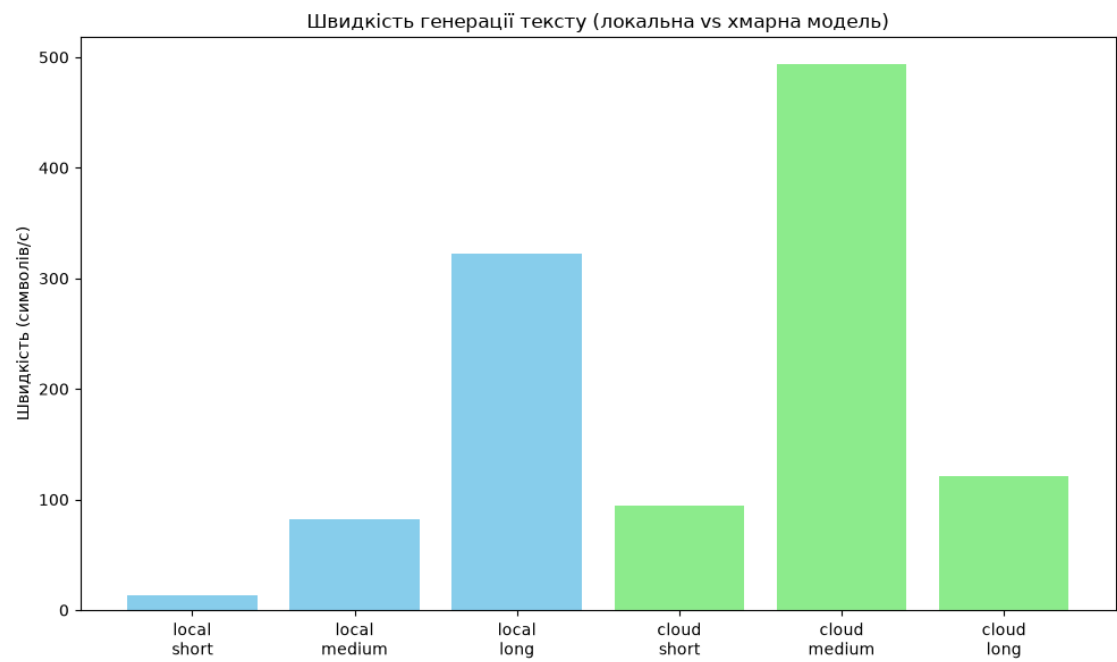


Рисунок 1 - Наочне порівняння локальної і хмарної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	14.520	2.086
Всього, с	17.810	2.234

Відповіді на питання:

- У якій моделі кращий TTFT і чому саме в неї? На коротких та середніх промптах набагато кращий TTFT має хмарна модель (0.2-0.3 с проти 2.0 с у локальній). Це пояснюється тим, що хмарна модель працює на потужних спеціалізованих серверах (LPU/GPU), які миттєво обробляють запит. Проте на довгому завданні TTFT хмари зріс до 20 секунд, бо це міркувальна модель (reasoning), яка витрачає час на внутрішні роздуми перед першим словом.
- Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть. Не завжди. Наприклад, на довгому промпті хмарна модель мала дуже поганий TTFT (20 секунд думала), але коли почала писати, її швидкість генерації

була все одно вищою, ніж у локальної. Тобто затримка перед стартом може бути великою, але сама "друкарська машинка" працює швидше.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого? На коротких запитах хмарна модель беззаперечно перемагала за всіма показниками. Але на довгому промпті (де треба було написати складну інструкцію) хмарна модель "задумалася" на 20 секунд, тоді як локальна почала відповідати майже так само швидко, як і раніше (2 секунди).

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску? Повтори відрізняються через затримки в мережі інтернет (пінг) та поточне навантаження на сервери Groq або на процесор мого комп'ютера. Тому одного запуску категорично недостатньо — для об'єктивної оцінки потрібно робити декілька запусків і брати медіану, щоб відкинути випадкові затримки.

5. Чому холодний старт повільніший за теплий? При холодному старті система змушена вантажити гігабайти файлів моделі з повільного жорсткого диска (чи SSD) в оперативну пам'ять (RAM). При теплому старті модель вже "висить" у швидкій оперативній пам'яті, тому обробка починається миттєво.

Питання до чат-бота:

1. Що фізично відбувається під час TTFT і чому цей показник вимірюють окремо?

Під час TTFT модель виконує етап «prefill»: вона паралельно зчитує весь вхідний промпт, токенизує його та прораховує контекст. Цей показник вимірюють окремо, бо саме TTFT відповідає за користувацький досвід — наскільки система здається «живою» і чуйною, незалежно від того, як швидко вона потім друкуватиме текст.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації?

Гірший TTFT у хмари виникає через час на передачу запиту через інтернет або через використання алгоритмів прихованих роздумів (reasoning). Але після появи першого токена етап генерації («decode») виконується на надпотужних серверних чипах, що генерують слова значно швидше за домашній комп'ютер.

3. Чому перший запуск локальної моделі повільніший і що таке холодний старт?

Холодний старт — це процес первинного завантаження файлу з вагами

нейромережі з диска (SSD/HDD) у пам'ять відеокарти або процесора. Наступні запуски (теплий старт) швидші, бо операційна система тримає модель «напоготові» в швидкій оперативній пам'яті.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків?

Медіана дозволяє відкинути аномальні відхилення (наприклад, раптовий стрибок пінгу інтернету або тимчасове завантаження фонові програми на локальному ПК). Середнє арифметичне сильно спотворюється від одного "випадкового" повільного запиту, а медіана показує реальний стан речей.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді? Обробка вхідного запиту (prefill) виконується паралельно — чип обчислює матриці для всіх слів запиту одночасно. Генерація відповіді (decode) є авторегресивною: кожне наступне слово генерується строго після попереднього (послідовно), що займає значно більше часу.

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 8 - Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель (qwen3.8-27b)
1. Факт	0	1	2
2. Формат	0	2	2
3. Стиснення	1	1	2
4. Код	2	2	2
5. Логіка	0	1	2
6. Українська	0	2	1
Разом /12	4	9	11

qwen3:4b краще знає українську і непогано працює з кодом, але все ще помиляється в логіці порівняно з гігантською хмарною моделлю

Таблиця 9 - Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель (qwen3.8-27b)
1. Факт	так	ні	ні
2. Формат	так	ні	так
3. Стиснення	так	так	ні
4. Код	так	так	так
5. Логіка	так	так	так
6. Українська	так	так	так

Відповіді на питання:

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця? Найбільше балів очікувано набрала хмарна модель qwen3.8-27b (11 балів з 12). Основна різниця виникла в завданнях на логіку (де локальні моделі не змогли вирішити математичну задачу з годинами) та на знання історичних фактів (де локальні вигадували випадкові дати кінопоказу, а хмарна дала точну відповідь).
2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі? Оцінки розійшлися на завданні «Стиснення тексту» (локальна модель то зберігала числа, то губила їх) та «Формат JSON» (в одному прогоні поставила крапку в кінці, що робить JSON невалідним). Це свідчить про низьку надійність малих моделей: через високу температуру генерації (0.7) їхні результати нестабільні, і вони не можуть гарантувати однакове виконання складних інструкцій при кожному запуску.
3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2? Так, хмарна модель ідеально виконала цю вимогу, згенерувавши чистий рядок тексту у форматі JSON без жодних додаткових блоків ``json чи форматувань.
4. Чи були проблеми з українською мовою - русизми, кальки, перехід на іншу мову? У локальної моделі (llama3.2:3b) проблеми були катастрофічними: вона генерувала неіснуючі слова з латинськими літерами («verdadeджnessi»),

створювала дивні кальки («Струма narrativa»), а також вживала русизми («зажорено»). Хмарна модель володіла ідеальною літературною українською мовою.

Приклад провалу

Найпоказовіший випадок: Математичне завдання на логіку (модель Папа3.2:3b).

Повний текст запиту:

Зал 180 місць, сеанс 1 год 45 хв + 15 хв перерва, робота з 10:00 до 23:00. Скільки сеансів і глядачів?. Покажи хід розв'язання.

Повний текст відповіді, без скорочень (уринок з 2-го прогону):

Пусть x — кількість сеансов, а y — кількість глядачів Площадь залу = $x * y = 180$

Можем записати цю проблему в математичному вигляді: $xy = 180$ 1 год 30 хв = 90 хвилин, які в часах = $(90/60) = 1,5$ год...

Яка правильна відповідь мала бути: Сеанс разом із перервою триває 2 години.

Робочий час (з 10:00 до 23:00) становить 13 годин. Повних сеансів у графік поміститься рівно 6 (до 22:00). Отже, глядачів буде: $180 * 6 = 1080$.

Пояснення, чому так сталося: Модель загубилася в багатокроковій арифметиці і втратила контекст фізичної реальності. Вона почала використовувати завчені з інтернету шаблони розв'язання базових шкільних рівнянь (пошук X та Y), абсурдно застосувавши формулу розрахунку площі до кількості глядачів. Також вона не змогла правильно виконати перехід між годинами та хвилинами, бо малі моделі дуже погано працюють з арифметикою через особливості токенизації чисел.

Питання до чат-бота:

1. Які ще підходи до оцінювання якості мовних моделей використовують на практиці? Використовують автоматизовані стандартизовані бенчмарки

(наприклад, MMLU для перевірки загальних знань, HumanEval для кодування), де моделі проходять десятки тисяч тестів. Також популярним є ELO-рейтинг на основі сліпих голосувань реальних користувачів (LMSYS Chatbot Arena) та автоматичне оцінювання відповідей однієї моделі іншою, більш потужною (LLM-as-a-judge).

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON? Під час фінального етапу тренування (RLHF) людей-оцінювачів вчили нагороджувати модель за красиве і читабельне оформлення коду в блоки markdown. Ця поведінка настільки глибоко «вшита» у ваги моделі як «найбажаніша», що перебити її простим промптом буває складно, особливо для малих моделей, які гірше розуміють заперечення (слово «без»).

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською? Англійські тексти складають понад 90% навчальної вибірки (датасету) для ШІ, тоді як частка українських текстів дуже мала. Оскільки малі моделі мають обмежену «ємність мозку» (кількість параметрів), вони фіксують переважно англійську граматику і структуру. Під час генерації українською вони часто просто на льоту перекладають власні англійські думки, що призводить до кальок та помилок.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску? Це відбувається через параметр «температури» (у нашому коді `temperature=0.7`). Модель не вибирає єдине абсолютно правильне слово, а обирає його випадковим чином з топу найбільш імовірних. Через це при кожному запуску формується інший шлях генерації тексту, який може виявитися як дуже вдалим, так і абсурдним.

5. Що таке галюцинація мовної моделі і чому модель радше вигідає факт, ніж визнає незнання? Галюцинація — це генерація моделлю впевненої, але абсолютно хибної інформації (як рік народження кіно 1917 замість 1895). Моделі не є базами даних і не мають функції «сумніву» — вони лише передбачають наступні слова на основі ймовірностей. Їх тренують так, щоб завжди надавати корисну відповідь,

тому при нестачі інформації вони завжди прагнуть продовжити речення, компілюючи схожі за контекстом, але неправильні факти.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 9 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	5
temperature	0.7	5	-
temperature	1.5	4	5
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

Таблиця 10 – Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання:

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому? Я поставлю temperature = 0.0, оскільки для генерації коду чи формату JSON потрібна максимальна передбачуваність і детермінованість. Будь-яка "креативність" моделі може призвести до порушення синтаксису JSON (зайва кома, текст замість ключа), що зламає парсер.

2. Яке — генератору варіантів заголовків? Для творчих завдань, як-от генерація заголовків чи назв, я встановлю `temperature` від 0.7 до 1.2. Це дозволить моделі видавати різноманітні, цікаві та неочевидні варіанти замість того, щоб раз по раз повторювати одне й те саме "найбезпечніше" слово.
3. Чим вплив `top_p` відрізняється від впливу `temperature` за характером, а не за силою? `Temperature` змінює контрастність всього розподілу: вона робить різницю між популярними та непопулярними словами меншою, підвищуючи шанси випадкового слова. А `top_p` жорстко відсікає всі малоймовірні варіанти ("хвіст" розподілу), залишаючи лише ті токени, сумарна ймовірність яких дорівнює заданому відсотку, після чого вибір відбувається лише серед цього обмеженого кола.
4. Чи означає `temperature=0` повну відтворюваність результату? Що показав ваш експеримент із 10 запусків? Не завжди. В експерименті з 10 запусків модель видала однакові відповіді, але під час творчого тесту (5 запусків при `temp=0.0`) модель все ж видала 2 унікальні варіанти (`PocketPulse` та `SpentPulse`). Це пов'язано зі специфікою обчислень з плаваючою комою на відеокарті чи паралелізмом потоків, через що навіть при нульовій температурі можливі мікро-похибки, які змінюють обраний токен.
5. Наведіть найбезглуздішу відповідь, яку модель видала при `temperature=1.5`, і поясніть, чому вона взагалі могла з'явитися. Окремих абсурдних відповідей не було (модель видала `SpendHive`, `PennyPath`), але при `temp=0.7` вона раптом видала просто слово «Сумка». Це сталося тому, що через підвищену температуру шанси "зрозумілих і типових" слів зменшилися, а ймовірність випадкових, рідкісних слів з "хвоста розподілу" штучно підскочила, і модель обрала одне з них.

Питання до чат-бота:

1. Що саме математично робить параметр `temperature` із розподілом ймовірностей наступного токена? Він ділить значення логітів (сірі бали для кожного слова перед функцією `softmax`) на значення температури (T). Якщо $T > 1$, логіти

зменшуються і "зближуються", через що графік ймовірностей стає більш плоским. Якщо $T < 1$, логіти збільшуються, і ймовірність найпопулярнішого токена стрімко наближається до 100%.

2. Чим `top_p` відрізняється від `temperature` — обидва впливають на різноманітність, але за яким механізмом? `Temperature` діє на всі токени у словнику, змінюючи їхню "вагу". Натомість `top_p` (nucleus sampling) сортує токени за ймовірністю і відкидає всі слова, що знаходяться після межі (наприклад, 90%). Механізм `top_p` — це жорсткий фільтр, а `temperature` — це балансувальник.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне? Офіційна рекомендація більшості провайдерів (OpenAI, Groq) — змінювати лише один з параметрів за раз (або `temperature`, або `top_p`), залишаючи інший за замовчуванням (зазвичай 1.0). Спільне їх використання призводить до непередбачуваної поведінки генерації.

4. Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках? Обчислення на GPU використовують матричні множення з рухомою комою (`float16` / `bfloat16`), які через оптимізації і паралелізм можуть давати мікроскопічні похибки округлення. На межі, коли два токени мають майже ідентичну ймовірність (наприклад, 49.999% і 50.001%), цей шум може змінити лідера, що спричинить розгалуження всього подальшого тексту.

5. Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту? Для JSON, витягу сутностей, написання коду та аналітики використовують `temperature = 0.0`. Для чат-ботів технічної підтримки та фактчекінгу — від 0.2 до 0.5. Для маркетингових текстів, ідей, віршів та брейнштормінгу — від 0.7 до 1.0 (а іноді й до 1.5).

Висновки та Підсумкова таблиця

Для наступних лабораторних робіт, де модель повинна віддавати структуровані дані, я завжди буду встановлювати `temperature = 0`. Налаштування за замовчуванням (зазвичай 0.7) є абсолютно непридатним, адже навіть невелика креативність моделі гарантовано призведе до порушення формату (зайві символи, коментарі в коді, спотворення ключів), що призведе до падіння всієї програми при парсингу JSON.

Таблиця 11 - Підсумкове порівняння моделей

Критерій	Локальна мала (llama3.2:3B)	Локальна більша (qwen3:4B)	Хмарна (qwen3.8-27b)
Швидкість генерації	Середня (13-80 симв/с)	Добра	Дуже висока (до 490 симв/с)
TTFT	Швидко (~2 с)	Швидко (~2 с)	Миттєво для коротких (0.2 с), довго для логіки (20 с)
Якість, балів /12	4	9	11
Стабільність формату	Дуже низька	Добра	Ідеальна
Потрібні ресурси	Мінімальні (2 ГБ RAM)	Середні (3-4 ГБ RAM)	Нульові (навантаження на сервер)
Вартість	Безкоштовно	Безкоштовно	Безкоштовно в лімітах API / Платна
Куди йдуть ваші дані	Нікуди (залишаються на ПК)	Нікуди (залишаються на ПК)	На сервери провайдера (Groq)
Працює без інтернету	Так	Так	Ні

Коли обирати саме її	Тільки для розваги або тестування середовища, коли немає інтернету.	Для простих скриптів, асистентів коду та завдань, що вимагають повної приватності даних (комерційна таємниця, особисті дані).	Для складної логіки, розробки розумних агентів, надійного парсингу JSON та важких завдань, коли дані не є секретними.
----------------------	---	---	---

Під час виконання цієї лабораторної роботи я не лише ознайомила з базовими принципами роботи великих мовних моделей, але й на практиці стикнулася з реальними інженерними викликами, які виникають при розробці ШІ-агентів. Найголовнішим відкриттям стало те, наскільки складно змусити модель стабільно видавати машинозчитуваний формат. Попри прямі інструкції, моделі постійно намагалися обгорнути JSON у Markdown-блоки, що миттєво ламало парсери. Це наочно довело, що для програмної інтеграції промптів недостатньо — необхідні жорсткі системні обмеження формату та обов'язкова фіксація параметра температури на нулі для уникнення зайвої "креативності".

Цікавим і дещо несподіваним викликом стала робота з хмарним API. Під час тестування потужної міркувальної моделі від Gpt я зіткнулася з помилкою 429 RateLimitError. З'ясувалося, що перед тим як видати фінальну відповідь, такі моделі генерують величезний прихований ланцюжок думок. Ці "невидимі" токени не лише роблять показник TTFT надзвичайно довгим (іноді до 20 секунд), але й блискавично вичерпують ліміти безкоштовного тарифу. При виборі архітектури агента критично важливо балансувати між розумністю моделі та реальними квотами на токени.

Окрім чисто архітектурних нюансів, робота з LLM підкинула й суто технічні проблеми, такі як падіння скриптів через помилки кодування (UnicodeEncodeError) у консолі Windows під час виводу згенерованого тексту. Будь-який сирий вивід від нейромережі потрібно відразу зберігати у файли з

примусовим кодуванням UTF-8, оскільки моделі можуть генерувати непередбачувані спецсимволи.

У підсумку, порівняння локальних та хмарних моделей показало чіткий розподіл ролей. Локальні рішення чудово підходять для гарантії приватності, обробки простих текстів та базового коду, але коли йдеться про складну математичну логіку чи бездоганне дотримання синтаксису, хмарні моделі залишаються незамінними. Я переконалася, що розробка ШІ-інструментів - це постійна робота з непередбачуваністю, якою необхідно навчитися керувати через правильні налаштування вибірки (temperature, top_p) та грамотний інжиніринг.